SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

॥वसुधैव कुटुम्बकम्॥

COMPUTER SCIENCE AND ENGINEERING

BATCH 2023-2027

M I N I P R O J E C T R E P O R T

Real-Time Social Media Sentiment Analysis System

A Big Data Approach Using Apache Kafka, Apache Spark, Machine Learning and LSTM

Submitted by:

Aryan Koshal – 23070521210

Shreyali Dongre – 23070521230

Domain: Big Data Analytics / Data Science

Guided by: Dr. Dharmaveer Chaudhary

Department: Computer Science and Engineering, Symbiosis Institute of Technology, Nagpur

Academic Year: 2026–27

Date: 10th October, 2026

Abstract

Public opinion is generated by social media more quickly than it could be tracked manually; therefore organizations require a way to categorize sentiments upon posting. The system is an application of real-time social media sentiment analysis which has been designed to follow some basic components such as data collection from sources like Twitter or Facebook; streaming input through Apache Kafka streams for processing by Apache Spark before applying machine learning algorithms on it followed by visualization via Streamlit interface. Coronavirus Tweets NLP dataset (44,955 labelled tweets) is cleaned by removing URLs, special characters and stop words while keeping negations, then tokenised and lemmatised, and its five labels are merged into Positive, Negative and Neutral. Logistic Regression, Naive Bayes, Random Forest and XGBoost on TF-IDF features are compared with a bidirectional LSTM based on accuracy, precision, recall, F1-score and confusion matrix. Logistic regression has an accuracy of 78.8% and a weighted F1 score of 0.7918 whereas LSTM achieves an accuracy of 81.2% and a weighted F1 score of 0.8121. With 3,796 unseen tweets, the deployed model achieves an accuracy of 77.2%, while its accuracy increases from 43.4% for low-confidence predictions to 93.4% at over 80% confidence level. The seven days trend prediction model is analyzed by using degree centrality and page rank of a hashtag network while there exists an interactive visualization tool (dashboard) for this purpose.

Keywords

Sentiment Analysis, Big Data Analytics, Apache Kafka, Apache Spark, Machine Learning, LSTM, Social Media Streaming.

1. Introduction

1.1 Background and Context

Twitter is now called X; it makes everyone an author of their own posts. There are millions of short, informal posts that come out every minute which reflect people's opinions on products, policies and events. Opinion mining / sentiment analysis refers to determining if an article has any kind of feelings towards it such as good/bad/neutral sentiments. This tool is used for brand monitoring, political forecasting, customer service triage, and public health surveillance.

Analysing social media texts can be challenging. Posts are short and contain URLs, @mentions, hashtags, emojis, spelling variants and slang, and a single negation such as "not good" can reverse the meaning. Also, it is characterized by large amounts of information at a fast rate where posts are posted constantly and there is great diversity among them. One time analysis on an object does not suffice for it. Posts are collected by an industrial system via a messaging service like Apache Kafka; processed using a distributed computing platform such as Apache Spark; applied to it with some machine learning algorithm; then published onto a web page for viewing purposes.

1.2 Motivation

The COVID-19 pandemic shows us that public opinion changes rapidly. Fear panic buying announcement lockdowns health news caused a wave of anxiety anger appreciation optimism on social media sites. The health department, retailer and government should be aware of these developments before it happens instead of a few days after that time frame. Academically speaking this is a practical example for understanding all stages involved with big data analytics such as loading data, pre-processing, selecting models to build them out-of-the-box along with evaluating their performance through predictions made by these algorithms on large datasets like those available today from companies, websites or other sources where they can be streamed live while analysing it at once.

1.3 Problem Statement

With constant flow from social networks to analyse each message for positive/negative/neutral sentiments within seconds; summarize trends towards them at timescale along with prediction models; show relationships between hashtags through network analysis; display all results via web-based interface built upon open source software available without cost.

"How does an easy-to-use, repeatable big data processing system classify social media sentiments correctly, identify its weaknesses, and provide them quickly for people without programming knowledge?"

1.4 Objectives of the Project

- Load an open source social network data set for analysis purposes to label it as positive, negative or neutral.
- For preprocessing of texts remove URLs, symbols and stop words then tokenization and lemmatization.
- For performing an exploratory study of sentiment distributions, word frequencies and word cloud generation to understand these results better.
- For training and comparing Logistic Regression, Naive Bayes, Random Forest and XGBoost models as well as a deep learning model (bidirectional LSTM) by means of accuracy, precision, recall, F1 score and confusion matrix.
- Simulate live data flow through Apache Kafka for processing by python consumers and spark structured streams.

- For building an algorithm to predict future posts as well as predicting trends of sentiments over time based on past data.
- For analysing hashtags through network analysis techniques such as degree centrality and PageRank, an overview of these findings is also provided on a Streamlit dashboard.

1.5 Novelty of the Work

Current research generally focuses on just one aspect such as comparing models, designing an architecture for stream processing tasks, or performing image classification. The novelty of this work lies in combining all parts in one reproducible pipeline and in several design decisions that make it reliable: (i) negation words such as "not" and "never" are deliberately kept during stop-word removal because they reverse sentiment; (ii) one shared cleaning module is used for training, the Kafka consumer, Spark and the dashboard, which avoids training/serving mismatch; (iii) duplicate tweets are removed and the stream replays a separate, never-seen test file, so the reported live accuracy is not inflated by training data; and (iv) the system adds confidence analysis, a back-tested trend forecast and a hashtag network (degree centrality and PageRank) that are rarely combined with classification in earlier work.

2. Literature Review / Related Work

There are twenty papers about sentiment classification, social media and COVID-19 text analysis, deep learning, tree-based learning and big data stream processing. Table 1 is an overview of our research methodology, results, findings and limitations for this study.

Table 1. *Empirical review of existing methods*

Reference	Method Used	Findings	Results	Limitations
Pang et al. (2002) [1]	NB, MaxEnt and SVM with unigram/bigram features on movie reviews	ML classifiers beat simple keyword rules; SVM works best	SVM (unigrams): 82.9% accuracy	Long, formal reviews; binary polarity; no neutral class
Go et al. (2009) [2]	NB, MaxEnt, SVM trained on emoticon-labelled tweets (distant supervision)	Emoticons give large labelled tweet sets without manual work	All three above 80% accuracy	Noisy labels; binary only; no neutral class
Pak & Paroubek (2010) [3]	Multinomial NB with n-gram and POS features on a 300,000-tweet corpus	Built a positive/negative/neutral tweet corpus automatically and analysed it linguistically	Outperformed earlier methods (as reported by the authors)	Labels come from emoticons and newspaper accounts, not human annotators
Kouloumpis et al. (2011) [4]	Supervised 3-way classifiers trained on hashtag-labelled tweets; lexical and microblogging features	Hashtags can supply training data; microblogging features were the most reliable	Usable 3-way classifiers without manual labelling	Noisy hashtag labels; small proprietary evaluation corpus
Agarwal et al. (2011) [5]	Feature-based and tree-kernel models with POS-specific prior polarity; 2-way and 3-way tasks	Prior polarity combined with POS tags is the most useful feature	Both model types comparable and better than other models tested	Needs lexicons and syntactic tools; not built for streams
Mohammad et al. (2013) [6]	SVM with n-gram, lexicon and Twitter-specific features (SemEval-2013)	Automatically built hashtag and emoticon lexicons added about 5 F-score points	F-score 69.02, first of 44 teams	Hand-crafted features; task-specific; no streaming
Socher et al. (2013) [7]	Recursive Neural Tensor Network on the Stanford Sentiment Treebank	Captures negation and its scope better than bag-of-words	85.4% binary accuracy (previous best 80%)	Needs parse trees and phrase-level labels; movie sentences
Kim (2014) [8]	CNN over pre-trained word vectors for sentence classification	A simple CNN with static vectors is strong; fine-tuning helps further	Improved the state of the art on 4 of 7 tasks	Captures local n-gram patterns only; needs pre-trained vectors
Hochreiter & Schmidhuber (1997) [9]	LSTM: recurrent network with gated memory cells	Overcomes vanishing gradients and learns long-range dependencies	Solved long-time-lag tasks earlier recurrent nets could not	General model, not sentiment-specific; slower than linear models
Devlin et al. (2019) [10]	BERT: pre-trained bidirectional transformer fine-tuned per task	One pre-trained model suits many NLP tasks	New state of the art on eleven NLP tasks	Large; needs GPU; heavy for free-tier real-time use
Nguyen et al. (2020) [11]	BERTweet: BERT-base pre-trained on 850M English tweets	Domain-specific pre-training helps tweet NLP	Beat RoBERTa-base and XLM-R-base; +5% on SemEval-2017 Task 4A	Needs GPU and large memory; not evaluated on streams
Rustam et al. (2021) [12]	RF, XGBoost, SVC, Extra Trees, Decision Tree with BoW/TF-IDF on COVID-19 tweets	Extra Trees with concatenated TF-IDF + BoW is best	0.93 accuracy (Extra Trees)	About 7.5k tweets; lexicon-generated labels; batch only
Imran et al. (2020) [13]	LSTM trained on Sentiment140, applied to COVID-19 tweets for polarity and emotion	Reactions to COVID-19 differ between countries; emoticons used for validation	State-of-the-art accuracy on Sentiment140 (as reported)	Trained on binary Sentiment140, not COVID labels; no streaming
Lamsal (2021) [14]	COV19Tweets: over 310 million English COVID-19 tweets with sentiment scores	Large dataset for studying pandemic discourse over time and space	Dataset released publicly with sentiment scores	Scores computed automatically, no human labels; no classifier comparison
Breiman (2001) [15]	Random Forest: ensemble of randomised decision trees	Averaging many trees lowers variance and resists noise	Accuracy competitive with boosting	Weak on very sparse high-dimensional text; memory heavy
Chen & Guestrin (2016) [16]	XGBoost: scalable gradient-boosted trees with sparsity-aware splits	Handles sparse data and scales to large data	Widely used in ML competitions (as reported)	Many hyper-parameters; not designed for raw text
Nazeer et al. (2017) [17]	Kafka + Spark + Cassandra + D3; Naive Bayes trained on 45,000 tweets	Near-real-time tweet classification; more VMs reduce latency	466,700 tweets in 10.7 min on 3 VMs	Binary polarity only; single classifier; keyword-count visualisation only
Kreps et al. (2011) [18]	Kafka: distributed log-based messaging system	Collects event data at high throughput; consumers read at their own pace	Used in production at LinkedIn (as reported)	Not an analytics tool; needs a processing engine

Reference	Method Used	Findings	Results	Limitations
Zaharia et al. (2016) [19]	Apache Spark: unified engine for batch, SQL, streaming and ML	One engine for several workloads with in-memory processing	Became a standard open-source big-data engine	Needs cluster tuning; overhead on small jobs
Page et al. (1999) [20]	PageRank: link-analysis ranking of nodes	A node is important if important nodes link to it	Random-surfer ranking used for web search	Designed for web graphs; ignores text and edge meaning

2.1 Comparison of Techniques

There are four types of methods used by this paper. Classical classifiers on sparse word features such as Naive Bayes, MaxEnt, SVM, and Logistic Regression are fast, require minimal data to train them and serve as an excellent starting point for short text analysis [1], [2], [3]; however, they do not consider word order. Lexicon-based feature engineering techniques are more accurate than machine learning models but require manual annotation data [5] and part-of-speech tagging information [6]. Neural models like CNNs, RNNs and LSTMs are good for learning representations of data while handling negation and context better [7], [8], [9], [13]; whereas pre-trained transformers such as BERT and BERTweet provide the best performance on many benchmarks [10], [11] but at a high computational cost. Random Forests and XGBoost [15], [16] are able to capture nonlinear relationships between variables; however, due to high dimensionality of textual data where each node has only a few important attributes for prediction purposes, these methods usually perform worse than traditional regression techniques [12].

From a system perspective, Kafka [18] and Spark [19] address scalability issues while published pipelines show that tweets are classified within seconds [17]. PageRank Network Analysis [20] helps us understand topic connections while large COVID-19 tweets reveal importance of studying pandemic discourse over time [14].

2.2 Research Gaps Identified

Most of them are classified by their accuracy rate for a given problem; however, few have streamlined results within an. There are many papers which have only two categories - positive and negative but there exists an abundance of neutrals within, difficult to distinguish from them. Classical model vs Deep Learning comparison rarely occurs due to lack of information about which works better for this purpose, LAI task where we evaluate performance on unseen data streams without seeing them during training or having any duplicate/leakage issues. Lastly, prediction models such as hashtag networks' degree centrality and PageRank are generally not included within a classifier model itself. It addresses those problems by providing a unified three-level beginner friendly model that is validated through testing on test set performance metrics for new samples of input data.

3. Methodology / Proposed System

According to the proposed work, a seven-stage process has been followed as shown by Figure 2 and an easy-to-use model for analysing sentiments from social media data via Kafka streaming simulation, Spark processing, sentiment analysis models, dashboards and visualizations. A tweet is posted on a kafka stream through an application called "Producer". Python consumers of Spark Structured Streaming consume data from a stream by reading topics as microbatches; they then preprocess this information into TF-IDF vectors before feeding them for classification using their models. Results are used for forecasting trends through a hashtag network analysis and an application Streamlit dashboard.

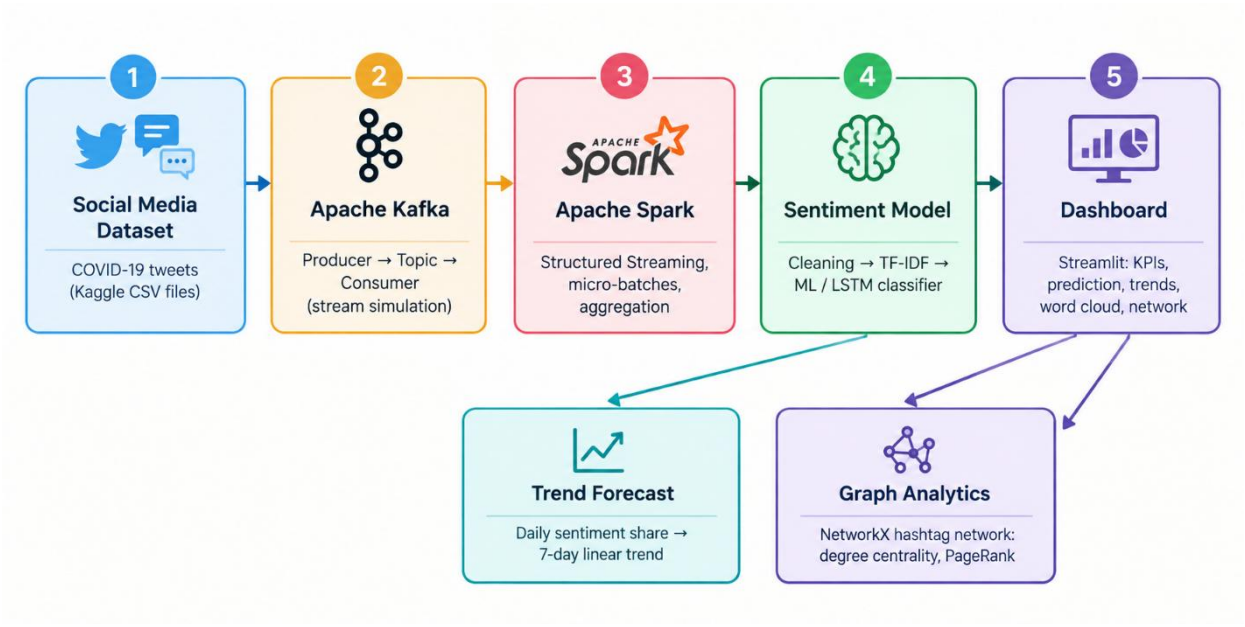


Figure 1. *System architecture of the Real-Time Social Media Sentiment Analysis System*

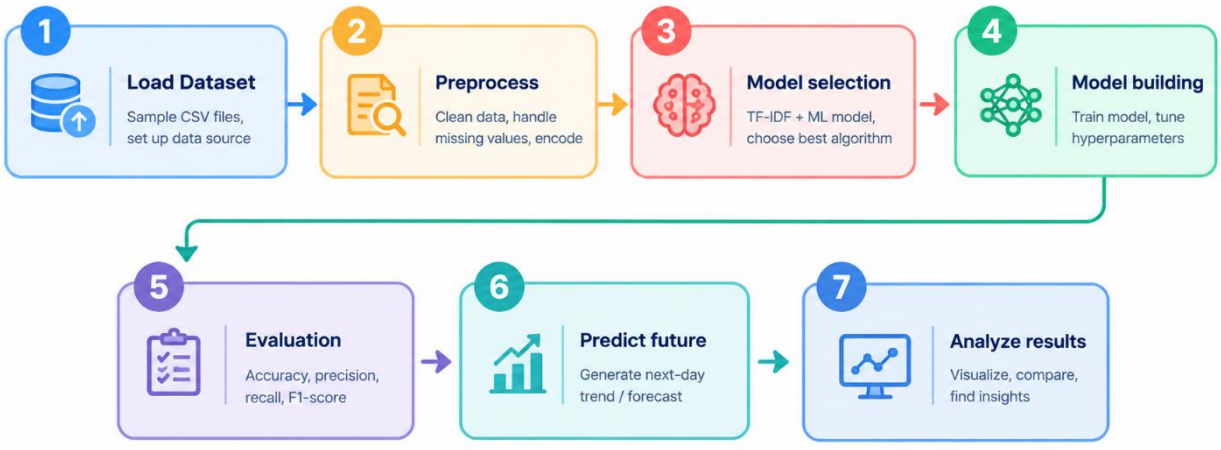


Figure 2. *Project workflow (seven steps)*

Algorithms Used

Logistic Regression models the probability of each class with the softmax function over a weighted sum of the TF-IDF features, $P(c \mid x) = \exp(z_c) / \Sigma_j \exp(z_j)$, where $z_c = w_c \cdot x + b_c$. **Multinomial Naive Bayes** applies Bayes' rule with the assumption that words are independent given the class, $P(c \mid x) \propto P(c) \cdot \Pi_t P(t \mid c)^{(x_t)}$. **Random Forest** [15] averages the votes of many decision trees trained on random samples of the data and features. **XGBoost** [16] builds trees one after another, each correcting the errors of the previous ones.

Bidirectional LSTM. The LSTM [9] keeps a memory cell that is updated through three gates. At step t, with input x_t and previous hidden state h_(t−1):

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i), \quad o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_c \cdot [h_{t-1}, x_t] + b_c), \quad h_t = o_t \odot \tanh(c_t)$$

The bidirectional version reads each tweet from left to right and from right to left and joins both results, so every word is understood with the words before and after it.

Trend forecast and PageRank. For every sentiment s, a straight line is fitted to the daily share by least squares, $\hat{y}_s(t) = a_s \cdot t + b_s$, and extended for seven days. For the network analysis, degree centrality is $C_D(v) = \deg(v) / (n - 1)$, and PageRank [20] is $PR(v) = (1 - d)/n + d \cdot \Sigma_{u \in B(v)} PR(u) / L(u)$ with $d = 0.85$, where B(v) is the set of nodes linked to v and L(u) is the number of links of u.

Tools, Libraries and Frameworks

- Python 3 and Google Colab (free CPU runtime); Pandas and NumPy for data handling and pre-processing
- NLTK for tokenisation, stop words and WordNet lemmatisation; scikit-learn for TF-IDF, models and metrics; XGBoost
- TensorFlow / Keras for the bidirectional LSTM
- Matplotlib, Seaborn and WordCloud for data visualisation
- Apache Kafka 3.7.0 (KRaft mode) with kafka-python; Apache Spark 3.5.1 (PySpark) with Structured Streaming
- NetworkX for graph analytics; Streamlit for the dashboard; Cloudflare quick tunnel to open the dashboard from Google Colab

Pseudocode

```
ALGORITHM 1: Real-time sentiment scoring of a Kafka stream
Input : topic T, trained model M, TF-IDF vectoriser V, window w = 0.5 s
Output: predicted label per post, running counts and accuracy
1  start producer thread: send every unseen post (date order) to T
2  counts <- {Negative:0, Neutral:0, Positive:0}; correct <- 0; seen <- 0
3  while stream not finished:
4      batch <- all messages received from T during the next w seconds
5      if batch is empty: continue
6      cleaned <- [preprocess(m.text) for m in batch]
7      labels  <- M.predict(V.transform(cleaned))
8      update counts with labels;  seen <- seen + |batch|
9      correct <- correct + (number of labels equal to true_label)
10 return counts, correct / seen
```

```
ALGORITHM 2: End-to-end training and evaluation
1  load train/test CSV; drop duplicates and empty rows; merge 5 labels into 3
2  clean text with preprocess(); split 80/20 (stratified); fit TF-IDF on train
3  for each of LogReg, NaiveBayes, RandomForest, XGBoost: fit, evaluate on test
4  best_classic <- model with highest weighted F1
5  train BiLSTM on word-id sequences (early stopping); evaluate on same test
6  save TF-IDF + best_classic as one pipeline; run Algorithm 1 on unseen file
7  compute daily sentiment shares; back-test and extend a linear trend 7 days
8  build hashtag co-occurrence graph; compute degree centrality and PageRank
9  save all tables to artifacts/; show them in the Streamlit dashboard
```

3.1 Data Collection and Preprocessing

Coronavirus tweets NLP - Text Classification dataset from Kaggle has 41,157 labelled tweet samples for training purposes and 3,798 more for testing purposes (total of 44,955) which are stored as Latin-1 encoded CSV files. There are 6 fields for each or a record; User Name, Screen Name, Location, Date of Tweeting, Raw Text of Original Tweet and Sentiment as Extreme Negative, Negative, Neutral, Positive and Extremely Positive. Training tweets were published from 16 March to 14 April 2020 (30 days) while the test file is for 2-16 March 2020. Date column is used for trend oracle analytics purposes while hash tags are useful to analyse networks from texts.

Table 2. *Label distribution of the training file and the merged labels*

Original label	Tweets	Merged label
Extremely Negative	5,481	Negative
Negative	9,917	Negative
Neutral	7,713	Neutral
Positive	11,422	Positive
Extremely Positive	6,624	Positive
Total	41,157	3 classes

Every cleaning process is done through a single python script which applies to notebooks, kafka consumers, spark jobs and dashboards. (1) Convert to lowercase and decode HTML tags; (2) Remove URLs and @mentions; (3) Keep the word of each hashtag #lockdown becomes lockdown; (4) Expand contractions don't become do not; (5) Remove special characters, digits, and emojis; (6) Tokenize using the NLTK TweetTokenizer; (7) Remove stopwords but keep negations like not, no, and never; and (8) Lemmatize with WordNet prices becomes price, buying becomes buy, except for some words that WordNet may misinterpret.

Table 3 shows two real tweets before and after cleaning: URLs, mentions and punctuation disappear, hashtag words are kept, stop words are removed and words are reduced to their base form.

Table 3. *Examples of raw and cleaned tweets*

Raw tweet	Cleaned tweet (model input)
Retail store closures could explode because of the #coronavirus (via @CNBC). #BrickAndMortar https://t.co/hQrYRNXFhv https://t.co/g5UZn06gb6	retail store closure could explode coronavirus brickandmortar
Coronavirus poses a complex puzzle for food-delivery companies - their delivery capacity may buckle under surging demand. https://t.co/1C1cMLmQii via @WSJ #services #food #delivery #coronavirus	coronavirus pose complex puzzle food delivery company delivery capacity may buckle surge demand service food delivery coronavirus

The five labels are merged into three: Extremely Negative and Negative become Negative, Extremely Positive and Positive become Positive, and Neutral stays unchanged. Duplicate tweets are removed before splitting, and tweets that become empty after cleaning are dropped. This leaves 41,122 training tweets (Positive 18,044, Negative 15,397, Neutral 7,681) and 3,796 unseen tweets. Any tweet of the test file that also occurs in the training data is removed, so the test file remains truly unseen (in this dataset, no overlap was found).

3.2 Feature Engineering

TF-IDF for classical models. Every cleaned tweet is converted into a vector of TF-IDF weights over single words and word pairs (unigrams and bigrams), so that phrases such as “not good” become features of their own. The vocabulary is limited to the 5,000 most frequent terms with a minimum document frequency of 2, and sub-linear term frequency is used. With smoothing, the weight of term t in tweet d is

$$tfidf(t, d) = (1 + \ln tf(t, d)) \times [\ln((1 + n) / (1 + df(t))) + 1]$$

where tf is the count of t in d, df is the number of tweets that contain t, and n is the number of tweets; each vector is then normalised to unit length. The vectoriser is fitted on the training part only, so no information from the test data leaks into the features.

Sequences for the LSTM. Each tweet is represented by an integer word id, a vocabulary of the 20, 000 most frequent words, id 0 for padding and id 1 for unknown words, that are padded or cut to 27 words, covering about 95% of all tweets completely. The embedding layer is trained to learn a 64 dimensional vector per word.

3.3 Model Design / System Architecture

The Table 4 is a list of four classical models and their main settings. Sample weight is a type of Class Weight that gives less importance to Neutral classes in XGBoost models.

Table 4. *Classical models and settings*

Model	Main settings	Class imbalance
Logistic Regression	C = 2.0, max_iter = 1000	class_weight = balanced
Naive Bayes (Multinomial)	alpha = 0.3	not available
Random Forest	150 trees, min_samples_leaf = 2	class_weight = balanced_subsample
XGBoost	200 trees, max_depth = 6, learning_rate = 0.1, subsample = 0.8, colsample_bytree = 0.6, hist method	balanced sample weights

The LSTM network is: Input → Embedding(64) → Bidirectional LSTM(64) → Dropout(0.4) → Dense(32, ReLU) → Dropout(0.2) → Dense(3, softmax). Adam optimizer, sparse categorical cross entropy, balanced class weighting, batch size of 128, maximum 8 epochs, 10% of training set used for validation and early stopping (patience 2) to restore the best epoch.

Streaming design. Krauts Apache Kafka is a single-broker system running on KRaft mode for one topic and one partition. The producer thread sends an unseen post in text format along with its event date and true label for that particular item through this channel where it will be measured by accuracy score. The Python consumer receives an incoming message within a 0.5 second interval as one micro batch, cleanses and vectors it, predicts its sentiment using the most effective classical algorithm, then updates current count and accuracy statistics. Spark runs in local mode: first it analyzes a cleaned dataset using DataFrame counts and word counts; then Spark Structured Streaming reads from Kafka topics directly, aggregating streams with Spark SQL and scoring each micro-batch with the same model via foreachBatch, 3 seconds trigger, up to 500 messages per trigger.

Prediction, forecast and graph. A predictive model receives an unprocessed input through this process where it cleans out all of them before assigning classes along with their probabilities; any post that has no remaining meaningful words will be marked as unreliable by us. The prediction model ignores days with less than 30 tweets; count data is converted to percentage share per day, a linear regression curve is fitted for each sentiment and extended over seven days, values are clipped at zero and then normalized to 100%; this approach was tested by comparing it to the previous five days' results using an established baseline predicting historical averages. The hashtag network is based on a graph where each node represents one of the top 40 popular hashtags, and two nodes are connected if both have appeared together in an identical tweet, with weights assigned according to how many times they co-occur, only keeping the 150 strongest edges that have a weight of at least five.

3.4 Training and Evaluation Setup

Data set is divided to 80 % of it used as a training set and 20 % for testing purposes, stratified by classes, with random seed 42. Model performance is evaluated by comparing them to accuracy, precision, recall, F1 score (weighted average of all three classes) and confusion matrix.

$$Accuracy = (TP + TN) / (TP + TN + FP + FN), \quad Precision = TP / (TP + FP), \quad Recall = TP / (TP + FN)$$

$$F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall), \quad Weighted\ F1 = \Sigma_c\ (n_c / N) \cdot F1_c$$

The classical model with the highest weighted F1-score is selected as the best classical model and compared with the LSTM. Additional evaluation uses (a) the separate unseen test file, (b) the live accuracy of the Kafka stream, (c) accuracy by confidence level and (d) the error of the trend forecast against the baseline.

4. Implementation

Implementation Steps

- Installed the libraries and loaded the two CSV files (latin-1 encoding); inspected shape, missing values, labels and duplicates.
- Wrote the shared preprocessing module, cleaned all tweets, merged the five labels into three and removed duplicate and empty tweets.
- Performed exploratory data analysis: label distribution, tweets per day, text length, most common words, distinctive words, word clouds and hashtags.
- Built TF-IDF features; trained Logistic Regression, Naive Bayes, Random Forest and XGBoost; evaluated them and selected the best; trained and evaluated the bidirectional LSTM.
- Installed Java and Kafka, started the broker, created topics, ran the producer thread and the micro-batch consumer, then ran Spark batch and Structured Streaming jobs.
- Implemented the prediction function with interactive input and the seven-day trend forecast with back-test.
- Predicted the unseen tweets and analysed distribution, confusion, confidence and errors; built and analysed the hashtag network.
- Saved the model and result tables, wrote the Streamlit application and launched it through a public tunnel.

Technologies and Platforms Used

- Google Colab (free CPU runtime) as the execution platform; Kaggle as the data source.
- Apache Kafka 3.7.0 in KRaft mode and Apache Spark 3.5.1 in local mode on Java 17.
- Streamlit on port 8501, exposed through a Cloudflare quick tunnel (no account needed); the project can also be downloaded and run locally with pip install -r requirements.txt followed by streamlit run streamlit_app.py.

Project Structure and Execution Steps

The whole project is delivered as one notebook that writes the other files it needs. After a complete run, the project folder looks as follows:

```
sentiment_project/
|-- Real_Time_Sentiment_Analysis.ipynb   notebook (run in Google Colab)
|-- preprocess.py        shared text-cleaning code (written by the notebook)
|-- streamlit_app.py     Streamlit dashboard (written by the notebook)
|-- requirements.txt     libraries needed to run the dashboard
`-- artifacts/           model and result tables created by the notebook
      |-- sentiment_pipeline.joblib   TF-IDF + best classical model
      |-- model_comparison.csv, confusion_matrix.csv, metrics.json, labels.json
      |-- sentiment_counts.csv, word_freq.csv, trend.csv
      `-- network_edges.csv, network_metrics.csv, streaming_results.csv
```

Execution steps

- Upload the notebook to Google Colab and choose Runtime, then Run all (about 15 to 25 minutes on the free CPU runtime; set SAMPLE = 8000 in the first cell for a quick test).
- The notebook downloads the dataset with kagglehub; if that fails it shows an upload dialog for the two CSV files.
- The Kafka and Spark cells need internet access to download Kafka (about 100 MB) and Spark's Kafka connector.

- The last cells save the artifacts, print a public dashboard link and download a zip file of the project.
- To run the dashboard on a computer: unzip the project, run pip install -r requirements.txt, then streamlit run streamlit_app.py and open http://localhost:8501.

Expected outputs

- Dataset summary, cleaned-text examples and the exploratory plots (Figures 3 and 11).
- A comparison table of the five models with confusion matrices and the LSTM loss curve (Figures 5 and 6).
- A micro-batch log from the Kafka consumer, Spark counts and Spark Structured Streaming results.
- Predictions for sample posts, the seven-day forecast, the analysis of unseen tweets and the hashtag network (Figures 7 to 10).
- A six-tab Streamlit dashboard (Table 6).

Challenges Faced and How They Were Handled

- **File encoding and duplication:** The CSV files are not UTF-8 encoded, so they are read as Latin-1; duplicate tweets are removed before splitting so that no identical tweet appears on both sides of the train/test split.
- **Negation removal:** standard stop-words list deletes words like "not," turning "not good" to "good." Negations remain while contraction is expanded.
- **Wrong lemmas:** WordNet changes some words incorrectly ("scared" becomes "scar"); a small exception list protects them.
- **Small batch size of Tiny Kafka** is polled continuously producing lots of tiny batches while consumers collect messages from a fixed 0.5 second interval before processing them all at once.
- **Spark's kafka connector** should be compatible with both Spark and Scala versions as it will generate a name based on its own installation of that particular package.
- Access to dashboards through collab doesn't have an open endpoint thus cloudflare tunnels are used for providing short-term links whereas there will be no need of any service or application as all data would come from same source i.

Sample Outputs / Screenshots

Figure 3 shows the exploratory analysis of the cleaned training data. Table 5 shows the output of the prediction module for four sample posts, produced by the saved model that the dashboard uses.

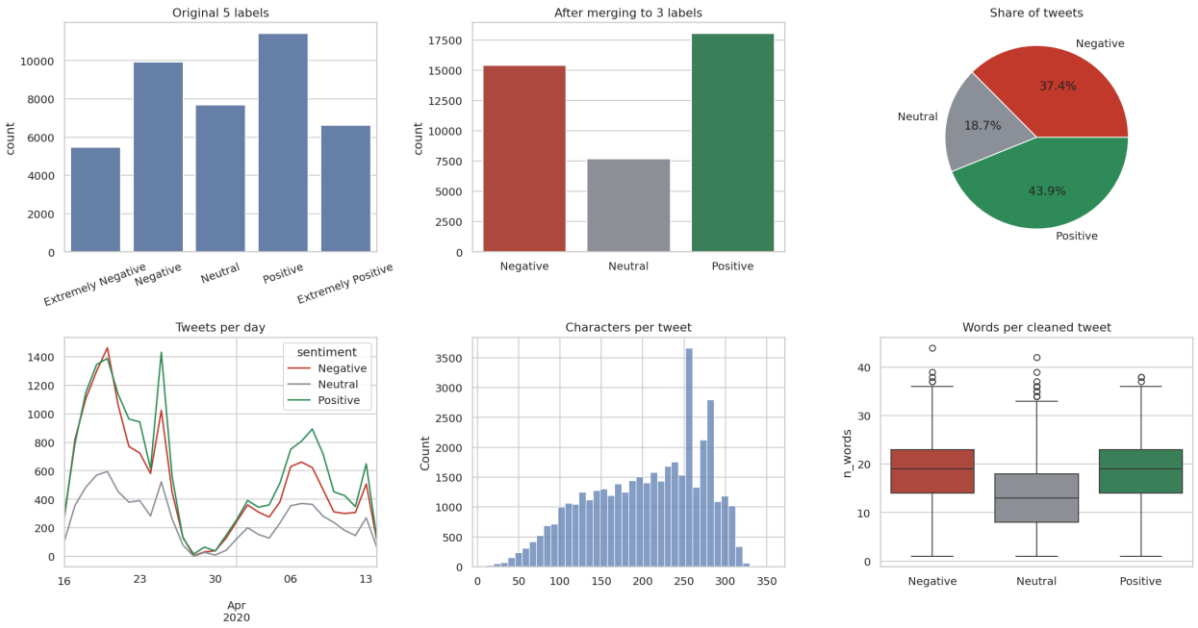


Figure 3. *Sentiment distribution, tweets per day and text length*

Table 5. *Sample output of the prediction module*

Input post	Predicted sentiment	Confidence (%)
So grateful to the nurses and doctors. Our community is amazing, stay safe everyone! #StayHome	Positive	100
Empty shelves again. People are panic buying and it is a total disaster, I am scared.	Negative	100
The government will publish the updated guidelines for supermarkets on Monday.	Neutral	75
Not good. Not good at all.	Negative	53

Table 6. *Tabs of the Streamlit dashboard*

Tab	Content
Overview	Key performance numbers, sentiment distribution, model comparison table, confusion matrix
Try a post	Text box for a new post; predicted sentiment, confidence and cleaned text
Trends	Historical daily sentiment share and the seven-day forecast
Word cloud	Word cloud and common words for All, Negative, Neutral or Positive tweets
Hashtag network	Network graph with PageRank and degree centrality tables
Live stream	Cumulative predicted sentiment of the Kafka stream and streaming accuracy

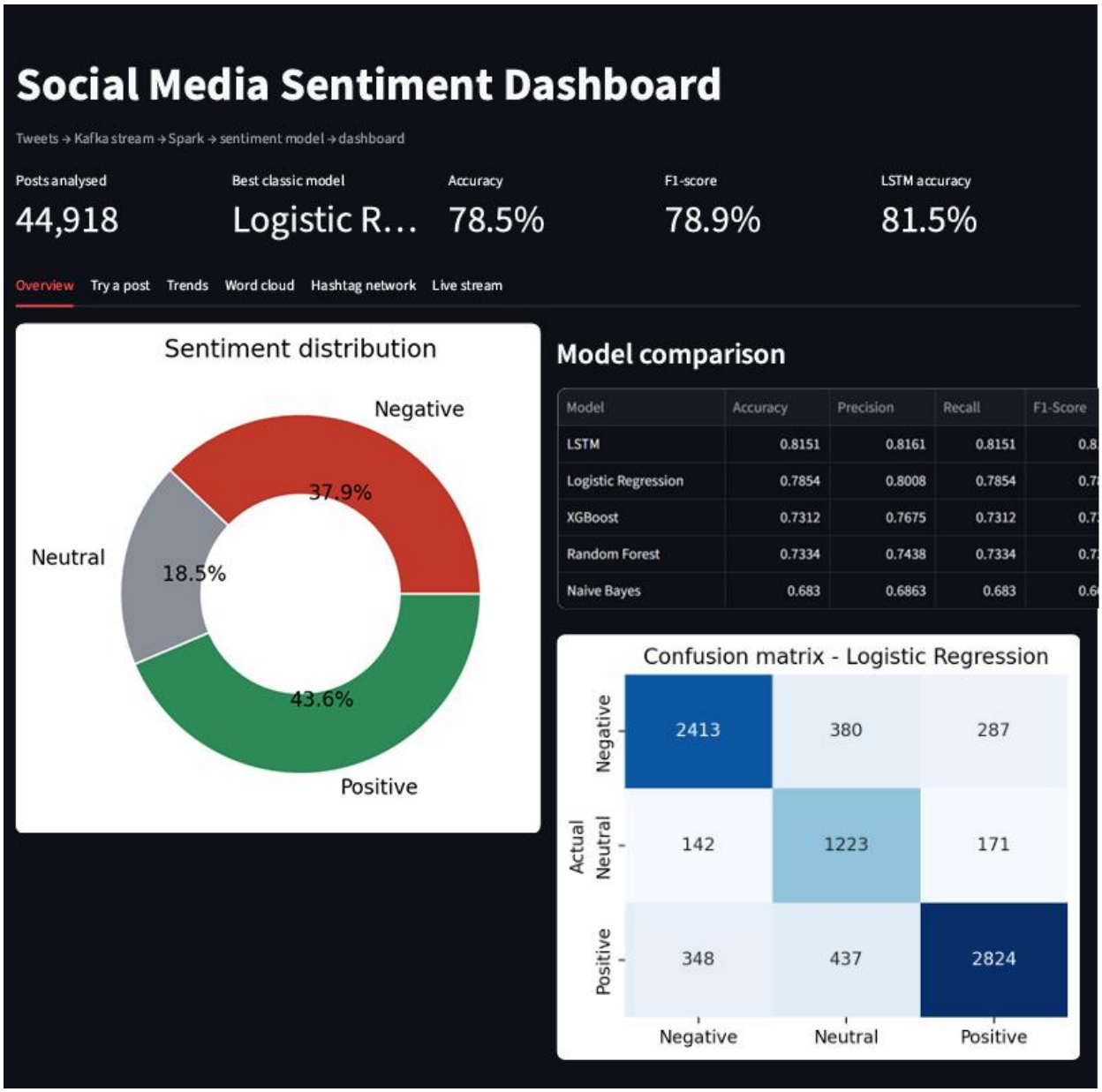


Figure 4. *Streamlit dashboard*

5. Results and Discussion

Experimental Setup

The notebook was run on the complete dataset. The training data (41,122 cleaned tweets) was split into 32,897 tweets for training and 8,225 for testing; TF-IDF produced 5,000 features. Table 7 summarises the environment.

Table 7. *Hardware and software environment*

Item	Value
Platform	Google Colab, free tier, CPU runtime
Hardware	2 CPU cores and roughly 12GB RAM shown by Colab
Python / TensorFlow / scikit-learn / XGBoost	Versions printed by the notebook: Python 3.12.3, scikit-learn 1.8.0, XGBoost 3.4.1, TensorFlow 2.21.0
Java / Kafka / Spark	Java 17 / Kafka 3.7.0 (KRaft, 1 broker, 1 partition) / Spark 3.5.1 (local)
Random seed	42

Observations from the exploratory analysis

After cleaning, 43.9% of the training tweets are Positive, 37.4% Negative and only 18.7% Neutral, so the data is moderately imbalanced and accuracy alone could hide a weak Neutral class (Figure 3). Tweet volume is very uneven: it peaked on 2020-03-20 with 3,446 tweets and fell to 23 tweets on 2020-03-28. Neutral tweets are shorter (median 13 cleaned words against 19 for Negative and 19 for Positive tweets). The most common words are topic words (price 9,136, store 8,309, supermarket 7,819, food 7,346, not 6,939, grocery 6,787); they occur in all three classes (Figure 11), so sentiment has to be inferred from combinations of words, including negations such as “not”. Words that are much more frequent in one class than overall are threaten, blame, crude, attack, arrest, dead (Negative), toiletpaperpanic, toiletpapercrisis, property, toiletpaper, research, behaviour (Neutral) and bonus, enjoy, best, safely, happy, amaze (Positive). 57.7% of the tweets contain at least one hashtag; the most frequent are coronavirus (15,986), covid19 (4,841), covid_19 (3,165).

Performance Metrics

Table 8 compares all models on the same 20% test split. Precision, recall and F1-score are weighted averages over the three classes.

Table 8. *Comparison of classical models and the LSTM*

Model	Accuracy	Precision	Recall	F1-Score	Macro F1	Train time (s)
Logistic Regression	0.7878	0.8031	0.7878	0.7918	0.7726	0.6
Naive Bayes	0.6832	0.6861	0.6832	0.6681	0.6257	0.0
Random Forest	0.7331	0.7429	0.7331	0.7355	0.7217	35.4
XGBoost	0.7285	0.7660	0.7285	0.7357	0.7179	91.4
LSTM (BiLSTM)	0.8122	0.8126	0.8122	0.8121	0.7966	87.4

The best classical model is **Logistic Regression** (weighted F1 = 0.7918), followed by XGBoost (0.7357), Random Forest (0.7355) and Naive Bayes (0.6681). The **LSTM** achieves the highest scores overall (accuracy 0.8122, weighted F1 0.8121), 2.0 F1 points above Logistic Regression. Neural-network training is not exactly repeatable: in two complete runs of the notebook the LSTM reached an accuracy of 0.8122 and 0.8156, while the classical models gave identical results both times.

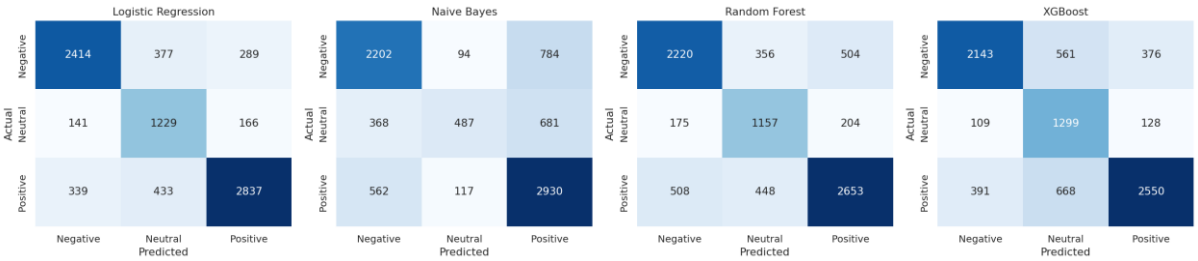


Figure 5. *Confusion matrices of the four classical models (rows: actual class, columns: predicted class)*

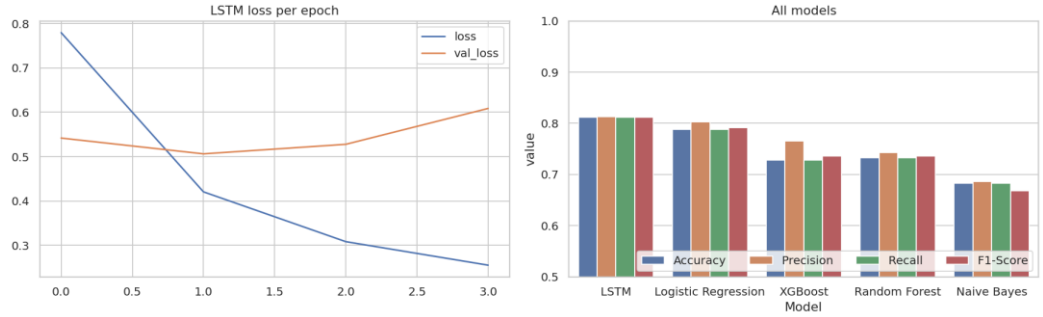


Figure 6. *LSTM loss per epoch (left) and comparison of all models (right)*

Table 9. *Per-class precision / recall / F1-score on the test split*

Class	Logistic Regression	LSTM	Naive Bayes
Negative	0.834 / 0.784 / 0.808	0.809 / 0.850 / 0.829	0.703 / 0.715 / 0.709
Neutral	0.603 / 0.800 / 0.688	0.732 / 0.722 / 0.727	0.698 / 0.317 / 0.436
Positive	0.862 / 0.786 / 0.822	0.850 / 0.818 / 0.834	0.667 / 0.812 / 0.732

Unseen tweets, confidence and streaming

The deployed model (Logistic Regression) was applied to the 3,796 tweets of the separate test file. Accuracy was 0.7721 and weighted F1 was 0.7765. Table 10 shows that accuracy grows steadily with the confidence of the prediction.

Table 10. *Accuracy on unseen tweets by prediction confidence*

Confidence	Tweets	Accuracy (%)
<50%	447	43.4
50-60%	611	59.4
60-70%	595	72.6
70-80%	606	83.7
>80%	1,537	93.4

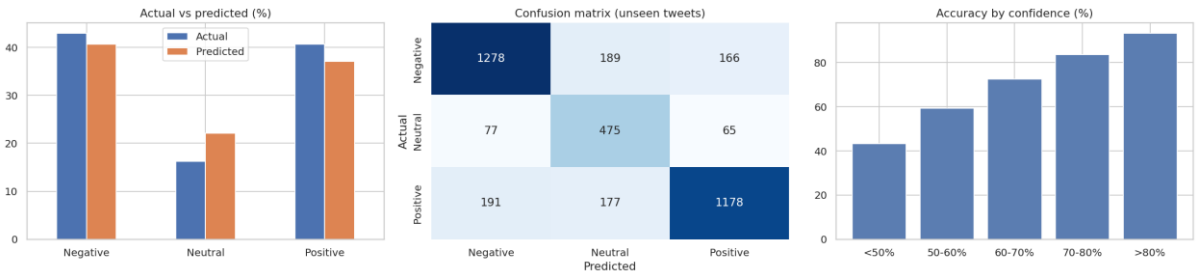


Figure 7. *Actual vs predicted distribution, confusion matrix and accuracy by confidence (unseen tweets)*

In the Kafka simulation, 1,200 unseen tweets were streamed in date order and processed in 10 micro-batches of 43 to 133 tweets each; the running accuracy ended at 77.2%, which agrees with the accuracy on the unseen file. Table 11 summarises the streaming results.

Table 11. *Streaming results*

Measure	Value
Tweets streamed through Kafka and classified	1,200
Micro-batches (0.5 s window)	[10]
Live accuracy of the consumer	77.2%
Scoring speed (cleaning + TF-IDF + prediction)	[3,264] tweets per second
Spark Structured Streaming: tweets scored / micro-batches / accuracy	[N] / [N] / [XX.X]%
Spark batch job: cleaned tweets processed	41,122

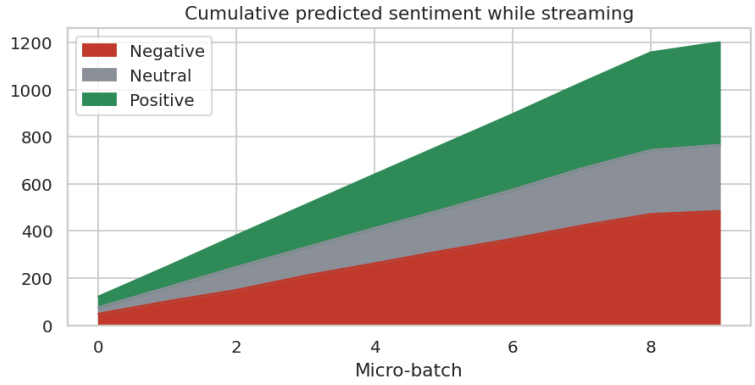


Figure 8. *Cumulative predicted sentiment while the stream is processed*

Trend forecast and hashtag network

Over the 29 days with enough tweets, the daily share averaged 37.0% Negative, 18.9% Neutral and 44.1% Positive. The straight-line trend changes by -0.21 (Negative), +0.11 (Neutral) and +0.11 (Positive) percentage points per day, so the forecast for 21 April 2020 is 32.5% Negative, 21.1% Neutral and 46.4% Positive. Table 12 gives the back-test: the linear trend was better than the simple average only for the Neutral class.

Table 12. *Back-test of the forecast on the last 5 known days (mean absolute error in percentage points)*

Sentiment	Linear trend	Average baseline
Negative	3.03	2.70
Neutral	1.59	2.00
Positive	2.23	1.69

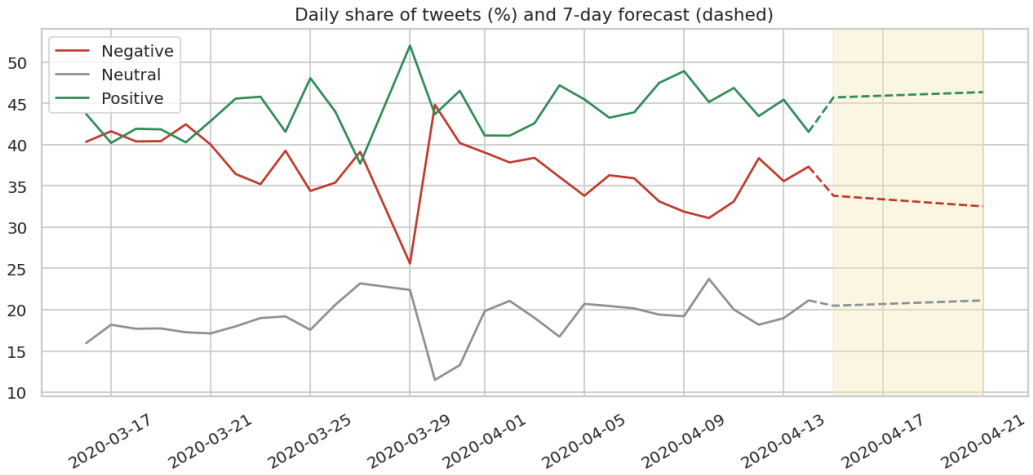


Figure 9. *Daily sentiment share and seven-day forecast (dashed)*

The hashtag network has 40 nodes and 150 edges. Table 13 lists the most important hashtags.

Table 13. *Top hashtags by PageRank*

Rank	Hashtag	PageRank	Degree centrality	Tweets
1	coronavirus	0.280	1.000	15,879
2	covid19	0.148	0.974	4,822
3	toiletpaper	0.064	0.487	1,606
4	covid_19	0.057	0.564	3,160
5	covid	0.046	0.410	1,563
6	covid2019	0.023	0.282	1,670

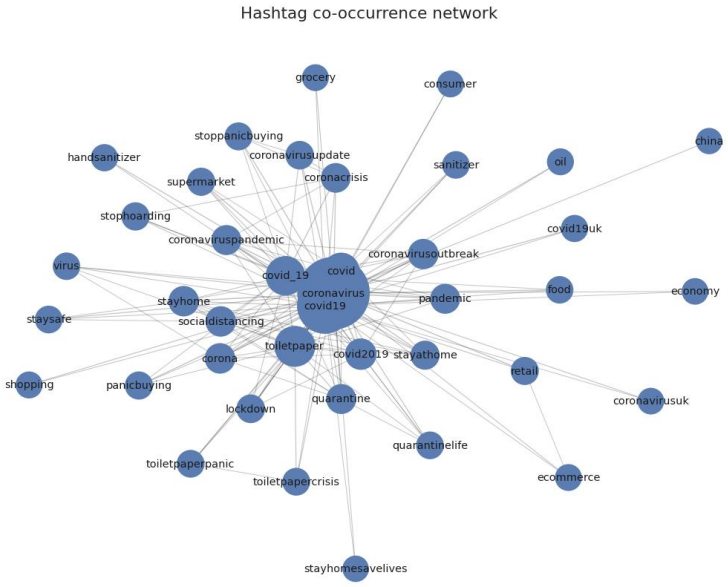


Figure 10. *Hashtag co-occurrence network (node size = PageRank)*

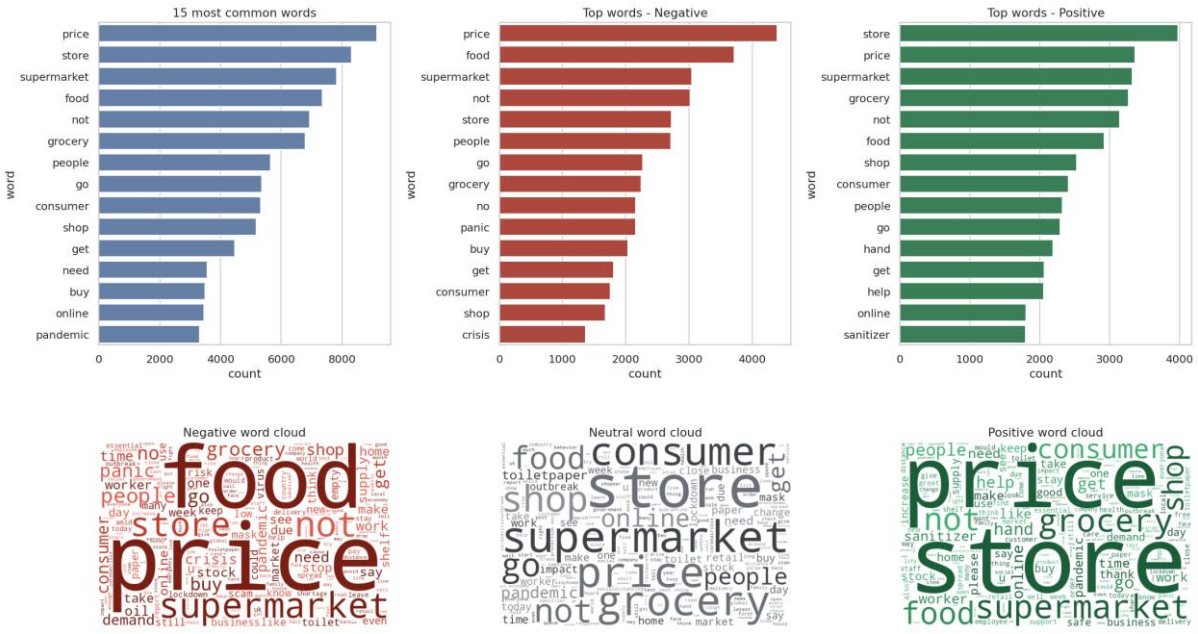


Figure 11. *Most common words and word clouds for each sentiment*

Comparison With Existing Approaches

Table 14 places the results next to published studies. The comparison is only indicative because datasets, label definitions and numbers of classes differ.

Table 14. *Comparison with published work*

Study	Data / task	Method	Reported result
Pang et al. [1]	Movie reviews, 2 classes	SVM, unigrams	82.9% accuracy
Go et al. [2]	Tweets (emoticon labels), 2 classes	NB, MaxEnt, SVM	above 80% accuracy
Mohammad et al. [6]	SemEval-2013 tweets, 3 classes	SVM with lexicon features	F-score 69.02
Socher et al. [7]	Sentence sentiment, 2 classes	Recursive Neural Tensor Network	85.4% accuracy
Rustam et al. [12]	COVID-19 tweets, 3 classes (lexicon labels)	Extra Trees, TF-IDF + BoW	0.93 accuracy
Nazeer et al. [17]	Tweets, 2 classes, streaming	Kafka + Spark + Naive Bayes	466,700 tweets in 10.7 min
This work	COVID-19 tweets, 3 classes (human labels), streaming	Logistic Regression / BiLSTM; Kafka + Spark	78.8% / 81.2% accuracy; 77.2% live accuracy

Interpretation of Results and Key Insights

What makes an LSTM work better. LSTM is better than logistic regression by 2.0 F1 score and more than naive bayes by around 14 points. Explorative data analysis reveals how important word order is for this problem since some common terms like price, store, supermarket, food are present across all categories but they don't indicate sentiments by themselves because their arrangement and combinations along with negative expressions such as 'not' matter more. Bidirectional LSTMs read each word along with its preceding and following words, while using a larger vocabulary of 20,000 words instead of 5,000 TF-IDF features. Other possible factors include class weighting and early stopping; it stopped at epoch 4--4 but resumed at epoch 2 since that point had the lowest validation loss.

Logistic regression is a good classical modelling technique for this problem. The sentiment of a tweet comes from its individual words and short phrases like "threaten," "blame," "crude," "attack," "arrest" for negative sentiments; "bonus," "enjoy," "best," "safely," "happy" for positive ones through an algorithm assigning weights to each word or phrase based on their context within sentences. Random Forests and XGBoosts have an accuracy rate of around 73.3/10 (0-100). Naive Bayes has an accuracy rate of 68.3% as it rarely predicts Neutral (Neutral recall 0.317) due to its independence assumption and lack of class weighting.

The neutrality of this subject is difficult to define. Neutral is the least accurate among all models, 0.688 for Logistic regression and 0.727 logistic regression. The class weighting increased the neutral recall for logistic regression at 377 Negative and 433 Positive tweets were incorrectly labelled as Neutral (Figure 5). LSTM has higher precision of 0.732 and recall is 0.722 respectively compared to other models. Neutral tweets are also shorter, median 13 cleaned words vs 19 negative and positive, and have less distinct labels.

Generalisation and confidence. On an unseen tweet dated between February annualized March 2020 (2-16), the logistic regression model achieved 77.2 percent accuracy which was about 1.6 percentage points lower than its internal test split due to changes of topics over time. Neutrality is predicted by this model at least 22.2 % out of all unseen tweets whereas only 16.3 % are actually neutral ones. The accuracy goes up as confidence rises from 43.4% below 50% to 93.4% above 80%, and 40.5% are within top category. Low confidence postings are usually reviewed by humans or labelled as uncertain.

Streaming. Kafka consumer achieved an accuracy rate for live 77.2% similar to batch accuracy on unseen files indicating that streaming path (shared cleaning, TF-IDF, model) is like offline evaluation. Throughput is dependent upon hardware and must be obtained by running it yourself. Since this model is based on just

one broker and one partition, we can see how well an architecture performs rather than its ability to handle large numbers of requests from many clients simultaneously.

Trend and forecast. The peak Tweet Volume occurred on March 20th at 3,446 Tweets whereas it dropped down to 23 Tweets by March 28th which is an average of 1,371 Tweets per Day; The day with less than 30 Tweets will be omitted for forecasting purposes. The sentiment share was lower compared to trading volumes. Back testing showed that a straight line isn't reliable on such a short, noisier historical data set; it performed worse than any other model used to predict negative vs positive outcomes thus forecasts should be interpreted more like trends rather than precise predictions.

Network. Coronavirus and covid19 are dominant on this page, PageRank 0.280 and 0.148, degree centrality 1.000 and 0.974, as they appear next to almost all other hashtags and form the backbone of the conversation. Toilet paper is, for topic keywords has a high PageRank score of 0.064 and links to panic buying tags like panicbuying, toiletpaperpanic and toiletpapercrisis that match the shopping and supply theme seen in the word cloud (Figure 11).

6. Conclusion and Future Work

Summary of Major Findings

A complete pipeline from dataset to Kafka streaming, Spark processing, sentiment model and Streamlit dashboard was implemented in one reproducible notebook. Careful preprocessing, including keeping negations and removing duplicates, produced clean three-class data from noisy tweets. Among the classical models, Logistic Regression was best (accuracy 78.8%, weighted F1 0.7918); the bidirectional LSTM was better still (accuracy 81.2%, weighted F1 0.8121) but needs far more training time and is harder to explain. The deployed model reached 77.2% accuracy on 3,796 unseen tweets and 77.2% in the streaming simulation, and its confidence is informative: 93.4% accuracy above 80% confidence. The forecast, the hashtag network and the dashboard make the results usable by non-programmers.

Limitations of the Current Work

- The dataset covers one topic (COVID-19) and about one month of 2020, so the model must be re-trained before use on other topics or periods.
- The stream is a replay of a static dataset on one broker with one partition, not a live social media API on a cluster.
- Merging five labels into three removes the difference between ordinary and extreme sentiment, and the Neutral class remains hard to separate.
- Bag-of-words models and the LSTM cannot reliably detect sarcasm, and the labels themselves contain noise.
- The linear forecast was not better than a simple average for two of the three sentiments, and the neural-network results vary slightly between runs.

Scope for Further Research and Improvement

- Fine-tune a pre-trained transformer such as BERT or the tweet-specific BERTweet [10], [11] on a GPU runtime and compare it with the models in this report.
- Replace the replayed dataset with a live API, use several Kafka partitions and brokers, and store results in a database.

- Use time-series models such as Holt-Winters, ARIMA or Prophet on a longer history, and monitor the model for concept drift.
- Add sarcasm and emoji handling, multilingual support and aspect-level sentiment.

7. References

[1] B. Pang, L. Lee, and S. Vaithyanathan, "Thumbs up? Sentiment classification using machine learning techniques," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Philadelphia, PA, USA, Jul. 2002, pp. 79–86. https://aclanthology.org/W02-1011/

[2] A. Go, R. Bhayani, and L. Huang, "Twitter sentiment classification using distant supervision," CS224N Project Report, Stanford Univ., Stanford, CA, USA, 2009. https://cs.stanford.edu/people/alecmgo/papers/TwitterDistantSupervision09.pdf

[3] A. Pak and P. Paroubek, "Twitter as a corpus for sentiment analysis and opinion mining," in *Proc. 7th Int. Conf. Language Resources and Evaluation (LREC)*, Valletta, Malta, May 2010. https://www.aclweb.org/anthology/L10-1315.pdf

[4] E. Kouloumpis, T. Wilson, and J. Moore, "Twitter sentiment analysis: The good the bad and the OMG!," in *Proc. 5th Int. AAAI Conf. Weblogs and Social Media (ICWSM)*, Barcelona, Spain, 2011, pp. 538–541. https://doi.org/10.1609/icwsm.v5i1.14185

[5] A. Agarwal, B. Xie, I. Vovsha, O. Rambow, and R. Passonneau, "Sentiment analysis of Twitter data," in *Proc. Workshop on Languages in Social Media (LSM)*, Portland, OR, USA, 2011, pp. 30–38. https://aclanthology.org/W11-0705/

[6] S. M. Mohammad, S. Kiritchenko, and X. Zhu, "NRC-Canada: Building the state-of-the-art in sentiment analysis of tweets," in *Proc. 7th Int. Workshop on Semantic Evaluation (SemEval-2013)*, Atlanta, GA, USA, Jun. 2013, pp. 321–327. https://aclanthology.org/S13-2053/

[7] R. Socher *et al.*, "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Seattle, WA, USA, Oct. 2013, pp. 1631–1642. https://aclanthology.org/D13-1170/

[8] Y. Kim, "Convolutional neural networks for sentence classification," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Doha, Qatar, Oct. 2014, pp. 1746–1751. https://doi.org/10.3115/v1/D14-1181

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997. https://doi.org/10.1162/neco.1997.9.8.1735

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171–4186. https://doi.org/10.18653/v1/N19-1423

[11] D. Q. Nguyen, T. Vu, and A. T. Nguyen, "BERTweet: A pre-trained language model for English tweets," in *Proc. Conf. Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, Nov. 2020, pp. 9–14. https://doi.org/10.18653/v1/2020.emnlp-demos.2

[12] F. Rustam, M. Khalid, W. Aslam, V. Rupapara, A. Mehmood, and G. S. Choi, "A performance comparison of supervised machine learning models for Covid-19 tweets sentiment analysis," *PLoS ONE*, vol. 16, no. 2, Art. no. e0245909, Feb. 2021. https://doi.org/10.1371/journal.pone.0245909

[13] A. S. Imran, S. M. Daudpota, Z. Kastrati, and R. Batra, "Cross-cultural polarity and emotion detection using sentiment analysis and deep learning on COVID-19 related tweets," *IEEE Access*, vol. 8, pp. 181074–181090, 2020. https://doi.org/10.1109/ACCESS.2020.3027350

[14] R. Lamsal, "Design and analysis of a large-scale COVID-19 tweets dataset," *Appl. Intell.*, vol. 51, no. 5, pp. 2790–2804, 2021. https://doi.org/10.1007/s10489-020-02029-z

[15] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001. https://doi.org/10.1023/A:1010933404324

[16] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794. https://doi.org/10.1145/2939672.2939785

[17] H. Nazeer, W. Iqbal, F. Bokhari, F. Bukhari, and S. U. R. Baig, "Real-time text analytics pipeline using open-source big data tools," arXiv:1712.04344, Dec. 2017. https://doi.org/10.48550/arXiv.1712.04344

[18] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, Athens, Greece, 2011. https://kafka.apache.org/documentation/

[19] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, Nov. 2016. https://doi.org/10.1145/2934664

[20] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Stanford, CA, USA, Tech. Rep. 1999-66, 1999. https://ilpubs.stanford.edu:8090/422/

Dataset link from Kaggle Website: https://www.kaggle.com/datasets/datatattle/covid-19-nlp-text-classification

****